

Contents

1. [Purpose and interpretation](#)
2. [Overview and principles](#)
3. [Scope](#)
4. [Classes](#)
5. [Attributes](#)
6. [Methods and signatures](#)
7. [Contracts for UML relationships](#)
8. [Evidence system](#)
9. [Attribute evidence consolidation](#)
10. [JSON model](#)
11. [PlantUML and Mermaid](#)
12. [Constructs not interpreted](#)
13. [Error handling](#)

1. Purpose and interpretation

This document describes the operation of UMLOOModeler 1.1. It shows how constructs found in object-oriented Python source code are represented in JSON, PlantUML, and Mermaid.

UMLOOModeler does not interpret every programming form offered by Python or convert them entirely into UML. The tool works with a delimited set of constructs and rules.

The absence of a relationship may mean that:

1. the code did not present evidence strong enough to support the relationship;
2. the evidence is strong, but the relationship was deliberately excluded from the version 1.1 contract;
3. the construct used is not recognized or requires a form of analysis that the tool does not support.

The absence of a relationship in the diagram only indicates that no rule accepted by the tool allowed it to be generated.

2. Overview and principles

2.1 Outputs

UMLOOModeler analyzes Python code and produces:

- structural and traceable JSON;
- a class diagram description in PlantUML;
- a class diagram description in Mermaid.

The tool identifies classes, attributes, methods, and a restricted set of relationships. Depending on the element, the JSON preserves name, type, line, expression, parameters, return type, source and destination classes, associated attribute, and nature of the evidence.

JSON is the recommended output for detailed inspection and auditing. The diagrams prioritize visualization and may omit traceability data available in the JSON.

2.2 Conservatism

The tool favors direct, delimited, and traceable constructs. Elements or relationships are not considered when their evidence is weak or outside the contract. Evidence is considered weak when it does not, by itself, demonstrate the existence of a persistent structural link between classes, as occurs with parameters that are not stored, method return types, and objects used only locally.

2.3 Minimalism and noise reduction

UMLOOModeler represents only a set of essential elements. Other possible UML features that could increase noise or inference, were kept outside the scope.

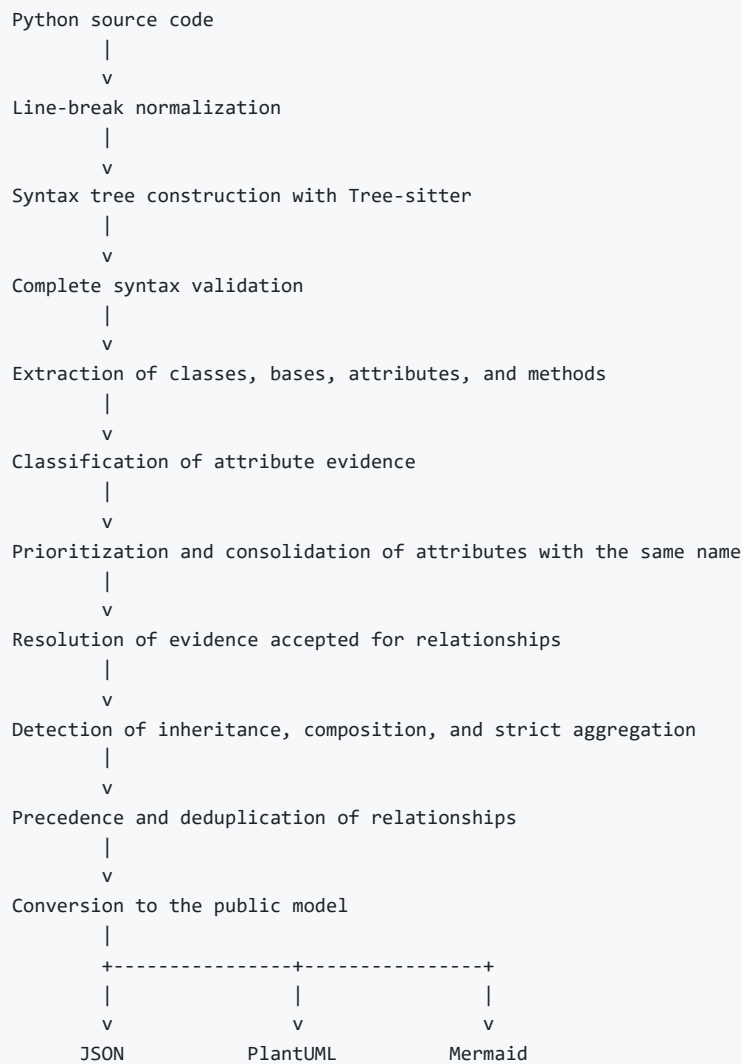
Examples of essential elements include:

- recognized classes, attributes, methods, and signatures;
- inheritance, composition, and strict aggregation when they meet the accepted rules;
- structural evidence and traceability data preserved in the JSON;
- representations of accepted structures in PlantUML and Mermaid.

2.4 Architectural and runtime evidence

UMLOOModeler produces architectural evidence derived from source code. This evidence does not prove that a method was executed and does not replace logs, traces, metrics, tests, expert evaluations, or other runtime evidence.

2.5 Processing



3. Scope

3.1 Input

UMLOOModeler analyzes one Python file at a time. The input must be syntactically valid and is limited to 1,000 lines of code.

3.2 Recognized elements

- Python classes;
- declared base classes;
- instance attributes assigned or annotated directly on self;
- regular, asynchronous, and decorated methods in the covered forms;
- simple and typed parameters;
- explicit return types;
- supported local inheritance;
- composition;
- strict aggregation;
- traceability data in JSON.

3.3 Supported relationships

Only three relationships are generated:

- inheritance;
- composition;
- strict aggregation.

Generic association, dependency, interface realization, and multiplicity are not generated.

3.4 Missing capabilities

UMLOOModeler does not perform joint analysis of multiple files or packages, resolve imports and aliases, perform general flow analysis, execute code, discover the return values of arbitrary calls, infer multiplicity, or prove the actual lifecycle of objects.

4. Classes

4.1 Declaration

A class declaration is recorded even when it is empty:

```
class Empty:
    pass
```

The tool preserves the extracted name, line, attributes, methods, and base classes.

4.2 Base classes

The bases field contains simple identifier names found syntactically in the class header:

```
class ElectricVehicle(Vehicle):
    pass
```

Structural result:

```
"bases": ["Vehicle"]
```

Qualified bases, such as `transport.Vehicle`, are not preserved in the `bases` field.

A base present in this list only generates inheritance if it also meets the contract described in Section 7.2.

4.3 Nested classes

A nested class is a class declared inside another class:

```
class OuterClass:
    class InnerClass:
        pass
```

In this example, the qualified form `OuterClass.InnerClass` identifies the inner class and the context in which it was declared. UMLOOModeler may find it and present it only under the simple name `InnerClass`. The tool does not fully implement this structure:

- it does not preserve the qualified identity `OuterClass.InnerClass`;
- it does not create a UML relationship solely because of nesting;
- it does not resolve `OuterClass.InnerClass()` as a simple local constructor;
- it keeps the attributes of the inner class separate from the outer class;
- it may present collisions between classes with the same name in different scopes.

This exclusion is deliberate. Full support would require qualified identities and should be addressed together with multiple files, packages, and conservative import resolution.

4.4 External classes

Imports may be read syntactically, but they are not resolved to create external classifiers or relationships.

5. Attributes

5.1 Recognized form

The recognized form requires an assignment or annotation on a direct attribute of `self`:

```
self.name = expression
self.engine: Engine
self.limit: int = 10
```

The attribute may be found in `init` or another method. The tool may preserve its name, type, line, expression, and `evidence_kind`.

The following are not accepted as direct attributes:

```
other.component = value
self.items[0] = value
```

5.2 Annotation and value origin

The annotation declares the type presented. The right-hand side records the origin of the value and may support a relationship:

```
self.service: BaseService = ConcreteService()
```

In this case, the explicit `BaseService` annotation defines the type presented for the attribute. The direct `ConcreteService()` call provides the evidence used for a possible composition with `ConcreteService`. Therefore, the type displayed for the attribute and the destination class of the relationship may be different.

An annotation without a value preserves the type but does not create a relationship:

```
self.service: Service
```

5.3 Literals

Types recognized in literals:

- integer: int;
- floating point: float;
- text: str;
- list: list;
- dictionary: dict;
- set: set;
- True and False: bool;
- None: NoneType.

5.4 Built-in constructors

Direct calls to `bool()`, `bytes()`, `dict()`, `float()`, `int()`, `list()`, `set()`, `str()`, and `tuple()` provide the corresponding built-in type when assigned to an unannotated attribute:

```
self.count = int()
```

When an explicit annotation is present, the annotation remains the type presented:

```
self.count: Number = int()
```

In both cases, the evidence is preserved as `call`. These built-in constructors do not generate UML relationships between local classes.

5.5 Unknown type

When no accepted rule provides a type, the attribute remains in the JSON with type equal to null and appears in the diagrams without a type suffix.

5.6 Expressions and references

Compound expressions are preserved without evaluation:

```
self.total = left + right
```

References are not assigned a type based on matching names:

```
self.value = source  
self.value = source.item
```

The relational exception is a typed `init` parameter stored directly, as described under strict aggregation.

5.7 Inner scopes

While searching for attributes, the attribute extractor does not assign to the outer class a self that belongs to an inner class, function, decorated definition, or lambda.

5.8 Visibility

In the diagrams:

- a name without an initial underscore is public and represented by +;
- a name with an initial underscore is private and represented by -.

This is a visual convention, not an enforcement of Python encapsulation.

6. Methods and signatures

6.1 Regular methods

Functions declared in the class body are recorded as methods. The tool preserves the name, recognized parameters, explicit return type, and line.

6.2 Accepted parameters

Simple parameters are preserved by name. Annotated parameters are preserved as name: Type:

Only a parameter named `self` is omitted from the generated signature. Other recognized parameters are preserved, including `cls`. Therefore, `cls` remains visible in a method decorated with `@classmethod` because UMLOOModeler does not interpret the decorator's meaning.

```
def train(self, epochs: int, verbose):  
    pass
```

Structural output:

```
train(epochs: int, verbose)
```

6.3 Default values and advanced parameters

UMLOOModeler recognizes the method itself, but omits parameters declared with default values, `*args`, or `**kwargs` from the extracted signature:

```
def configure(self, retries: int = 3, verbose=True):  
    pass  
  
def dispatch(self, *args, **kwargs):  
    pass
```

If these are the only parameters other than `self`, `params` is empty in the JSON, and no parameters appear in PlantUML or Mermaid. UMLOOModeler does not provide a complete guarantee for other combinations containing positional-only or keyword-only parameters.

6.4 Return type

Only the explicit annotation after `->` is preserved:

```
def count(self) -> int:
    return 0
```

The JSON records "return": "int". Without an annotation, it records "return": null. Although `init` returns `None` according to Python, it appears with null when `-> None` was not written.

Return types do not generate relationships.

6.5 Decorated methods

Decorated methods are preserved, but decorators are neither recorded nor interpreted as UML semantics. Evidence of the method's existence is strong; the absence of a special marker is a deliberate scope decision.

- `@staticmethod` does not add `{static}`;
- `@classmethod` does not create a special marker, and `cls` remains in the signature;
- `@abstractmethod` does not produce an abstract marker;
- `@property` remains a regular method;
- custom decorators do not create stereotypes;
- decorated `async def` is preserved without an asynchronous marker.

Interpreting decorators would increase complexity and could introduce noise, regressions, or incorrect inferences, especially for custom, combined, or parameterized decorators.

6.6 Visibility

In the diagrams:

- a method without an initial underscore is public and represented by `+`;
- a method with an initial underscore has package visibility and is represented by `~`.

Under this convention, `init` normally appears with `~`.

7. Contracts for UML relationships

7.1 General rule

An isolated type, parameter, return type, annotation, or name is not automatically converted into a relationship. Each of the three supported relationships requires its own contract.

7.2 Inheritance

Accepted form:

```
class Dog(Animal):
    pass
```

Criteria:

- the derived class is declared in the input;
- the base is written explicitly;
- the base is represented by a simple identifier;
- a recognized local class has the same name.

Multiple inheritance generates one generalization for each accepted local base:

```
class Duck(Swimmer, Walker):  
    pass
```

Qualified bases, such as `package.Base`, and imported external bases do not generate a relationship. A simple external base may remain in bases in the JSON, but it is not created as a classifier in the diagram.

Representation:

```
PlantUML: Derived -|> Base  
Mermaid: Derived --|> Base
```

7.3 Composition

Accepted form:

```
class Car:  
    def __init__(self):  
        self.engine = Engine()
```

Cumulative criteria:

- the left-hand side is a direct attribute of `self`;
- the entire right-hand side is a direct call;
- the call target is a simple identifier;
- the target matches a recognized local class;
- the destination class differs from the source class.

Composition may also be recognized outside `init`:

```
def install_engine(self):  
    self.engine = Engine()
```

This evidence is strong under the contract, but it does not prove that the method was executed or demonstrate the entire UML lifecycle at runtime.

If a different annotation is present, the declared type remains on the attribute, and the composition follows the constructed class:

```
self.engine: BaseEngine = ElectricEngine()
```

Attribute type: `BaseEngine`. Composition destination: `ElectricEngine`.

Representation:

```
Owner *-- Component
```

7.4 Strict aggregation

Accepted form:

```
class Car:
    def __init__(self, engine: Engine):
        self.engine = engine
```

Cumulative criteria:

- the value comes from an **init** parameter;
- the parameter has a type annotation;
- the right-hand side is exactly that parameter's identifier;
- the left-hand side is a direct attribute of **self**;
- the parameter type matches a recognized local class;
- the destination class differs from the source class.

The attribute name may differ from the parameter:

```
self.primary_engine = engine
```

A different attribute annotation does not change the origin of the aggregation:

```
def __init__(self, battery: Battery):
    self.power: Generator = battery
```

Attribute type: Generator. Aggregation destination: Battery.

The following do not generate aggregation: an untyped parameter, storage outside **init**, an intermediate variable, or insertion into a collection.

Representation:

```
Owner o-- Component
```

7.5 Relational precedence and deduplication

For the same ordered pair of classes:

```
composition > aggregation
```

Composition takes precedence regardless of whether it appears before or after aggregation in the code. Different attributes remain recorded in the JSON, although only one relationship line is shown for the same type, source, and destination.

When relationships have the same type, source, and destination, the first relationship evidence retained supplies the line, expression, and attribute in the JSON.

7.6 Generic associations and dependencies

UMLOOModeler does not represent generic association or dependency. References from other objects, isolated annotations, unstored parameters, local variables, objects that are only returned, method parameters, and return types do not produce relationships.

Annotated parameters and return types could support a syntactic dependency contract. Nevertheless, this support was deliberately excluded to keep the structural diagram concise and reduce noise.

7.7 Collections

UMLOOModeler does not derive relationships from collection elements, generic annotations, or collection operations:

```
self.books: list[Book] = []
self.books = [Book(), Book()]
self.books.append(book)
self.books.extend(books)
```

These forms do not produce association, aggregation, composition, or multiplicity. The analyzer does not traverse the inner elements of a collection or interpret append or extend as structural retention.

A local class that represents a container remains subject to the normal rules:

```
self.books = BookCollection()
```

This case may generate composition with BookCollection because a local class is constructed directly, not because the analyzer understood collection semantics.

7.8 Reflexive relationships

UMLOOModeler does not generate reflexive relationships.

Self-composition:

```
class Box:
    def add_inner_box(self):
        self.inner_box = Box()
```

Creation and retention constitute strong evidence, and the attribute is preserved as constructor_call. Even so, Box *-- Box is deliberately suppressed as a matter of scope and noise reduction. This is a known false negative; recursive relationships are valid in UML.

Reference to the instance itself:

```
self.reference = self
```

In the second case, no new object is constructed and no typed external object is stored. The assignment does not satisfy composition or aggregation. A reflexive association would be possible under another contract, but generic associations are not supported.

8. Evidence system

`evidence_kind` classifies the syntactic nature of the selected evidence. It is not a probability, confidence score, or proof of runtime execution.

evidence_kind	Minimal construct	Preserved information	Relationship effect
missing	<code>self.sensor: Sensor</code>	name, type, and line; expression null	none by itself
literal	<code>self.count = 10</code> Or <code>self.value = None</code>	expression and literal type	none
expression	<code>self.total = left + right</code>	expression and line; type generally null	none
reference	<code>self.copy = source</code> Or <code>self.item = source.item</code>	reference and line; type generally null	none without an additional contract
call	<code>self.component = factory.build()</code>	call and line; type generally null	none
constructor_call	<code>self.engine = Engine()</code>	constructed local class, expression, and line	may generate composition
typed_parameter_reference	<code>self.engine = engine</code> , with <code>engine: Engine</code> in <code>__init__</code>	stored typed parameter, expression, and line	may generate aggregation
base_class	<code>class Car(Vehicle)</code>	base, expression, and line in the relationship	may generate inheritance

An explicit annotation remains the type presented even when the right-hand side provides different evidence. `base_class` appears in relationships, not as the `evidence_kind` of an attribute.

Factories, functions, and arbitrary methods remain classified as `call`. The analyzer does not follow their bodies or execution to discover the return value; therefore, these calls do not support composition, aggregation, association, or dependency.

Recognized built-in constructors also use `call`, but they may provide a built-in type without generating a relationship between classes.

9. Attribute evidence consolidation

9.1 Consolidation rules

UMLOOModeler preserves at most one record for each attribute name in a class. When the same name occurs more than once, the extractor consolidates the evidence found. This process does not simulate runtime execution and does not simply preserve the last assignment.

The initial evidence priority, from highest to lowest, is:

1. a call whose target is a simple identifier, such as `Engine()` ;
2. other calls, literals, and references;
3. expressions;
4. a declaration without a right-hand-side value (`missing`).

A later occurrence with higher priority replaces the earlier record. Priority, however, is not the only selection criterion. When higher priority does not apply:

- a later explicit annotation may supplement an existing record that has no annotation;
- a later record with a known type may replace an existing record whose type is still unknown;
- a later record with a right-hand-side value may replace a declaration that has no value.

When none of these conditions applies, the first evidence found remains. The JSON therefore contains a consolidated record, not an attribute state history or a simulation of execution order.

9.2 Reassignment examples

Direct construction after `None` :

```
self.engine = None
self.engine = Engine()
```

The direct call has higher initial priority. `Engine()` is retained, later resolved as `constructor_call`, and composition is generated.

Direct construction before `None` :

```
self.engine = Engine()
self.engine = None
```

During extraction, the local class represented by `Engine()` has not yet been resolved as the attribute type. The later `None` literal provides the known type `NoneType` and replaces the still-untyped call under the missing-type rule. The analyzer consequently receives no constructor call to resolve, and composition is not generated.

Successive literal evidence:

```
self.status = None
self.status = "ready"
self.status = 100
```

These occurrences have the same priority, and the first record already provides a known type. The JSON therefore preserves the first literal, `None`.

An explicit annotation is retained as the type presented while the origin of the right-hand-side value remains available as relationship evidence:

```
self.service: BaseService = ConcreteService()
```

In this example, the attribute type is `BaseService`, while the direct `ConcreteService()` call may support composition with `ConcreteService`, as described in Section 5.2.

Relationship precedence and deduplication are separate later operations and are described in Section 7.5.

10. JSON model

10.1 Role of JSON

JSON is the most detailed structural model. It allows elements and relationships to be traced back to accepted constructs in the source code.

The public file contains:

```
{
  "classes": [],
  "relationships": []
}
```

10.2 Class

```
{
  "name": "Car",
  "attributes": [],
  "methods": [],
  "bases": ["Vehicle"],
  "lineno": 4
}
```

Fields:

- name: simple class name;
- attributes: extracted and deduplicated attributes;
- methods: extracted methods;
- bases: bases found syntactically;
- lineno: declaration line.

10.3 Attribute

```
{
  "name": "engine",
  "type": "Engine",
  "lineno": 6,
  "expression": "Engine()",
  "evidence_kind": "constructor_call"
}
```

Fields:

- name: name without the self prefix;
- type: accepted type or null;
- lineno: line of the selected evidence;
- expression: text of the right-hand side or null;
- evidence_kind: syntactic nature of the selected evidence.

A null type means that no safe type is available within the contract. It does not mean an error, Any, nonexistence of the attribute, or confirmation of a None value at runtime.

10.4 Method

```
{
  "name": "start",
  "params": "mode: str, retries",
  "return": "bool",
  "lineno": 8
}
```

Fields:

- name: method name;
- params: simplified signature of the recognized parameters;
- return: explicit return annotation or null;
- lineno: definition line.

10.5 Inheritance

```
{
  "type": "inheritance",
  "from": "Car",
  "to": "Vehicle",
  "lineno": 4,
  "expression": "Vehicle",
  "evidence_kind": "base_class"
}
```

10.6 Composition and aggregation

```
{
  "type": "composition",
  "from": "Car",
  "to": "Engine",
  "lineno": 6,
  "expression": "Engine()",
  "evidence_kind": "constructor_call",
  "attribute": "engine"
}
```

from is the class that declares the structure. to is the base, constructed class, or type of the received parameter, according to the relationship. These fields do not represent call or data flow.

attribute identifies the attribute associated with composition or aggregation. Inheritance does not use this field.

10.7 Versioning

JSON does not contain schema_version. Automated consumers must associate the format with the UMLOOModeler version and validate the fields before processing them.

11. PlantUML and Mermaid

11.1 PlantUML

```
@startuml
class "Car" {
+ engine: Engine
--
~ __init__()
+ start(): bool
}
Car *-- Engine
@enduml
```

Relationship notation:

```
Inheritance: Derived -|> Base
Composition: Owner *-- Component
Aggregation: Owner o-- Component
```

11.2 Mermaid

```
classDiagram
    direction TB
    class Car {
        +engine : Engine
        ~__init__()
        +start() bool
    }
    Car *-- Engine
```

Relationship notation:

```
Inheritance: Derived --|> Base
Composition: Owner *-- Component
Aggregation: Owner o-- Component
```

Differences in spacing, return notation, and inheritance arrows result from the syntax of each format and do not change the structural model.

12. Constructs not interpreted

This section summarizes the exclusions. The positive rules in the preceding sections constitute the primary contract.

12.1 Indirect flow

Intermediate variables are not followed:

```
temporary = Engine()
self.engine = temporary
```

The attribute may be preserved as reference, but no composition is generated.

12.2 Indirect and qualified calls

The return values of factories, functions, and arbitrary methods are not resolved. The same applies to qualified calls such as `package.Engine()` or `Outer.Inner()`. They may be preserved as call, without an invented type or relationship.

12.3 UML elements deliberately absent

- generic association and dependency;
- interface realization;
- optional or collection multiplicities;
- Enum as a UML enumeration;
- ABC and `abstractmethod` as abstract markers;
- Protocol as an interface;
- `final` as {leaf};
- properties, static members, and class methods as UML markers.

12.4 Structural scope

- attributes declared only in the class body are not supported instance attributes;
- imports, aliases, and external symbols are not resolved;
- code in other files is not analyzed jointly;
- classes with the same name may collide;
- runtime information is not collected;
- collections do not produce relationships with their elements;
- reflexive relationships are suppressed.

13. Error handling

13.1 Invalid syntax

If any part of the tree indicates a syntax error, the internal result is:

```
{
  "status": "error",
  "error": "Invalid Python syntax"
}
```

No partial model is produced. This decision avoids presenting as complete documentation a model derived from syntactically invalid code.

13.2 File without classes

A valid file without classes may produce empty lists. The absence of classes is not a syntax error.

13.3 Application errors

The applications may also reject a nonexistent file, non-.py input in the CLI, a file above the limit, a reading failure, overwriting without authorization, or a local renderer failure. These cases do not alter the parser rules.